CAPSTONE PROJECT REPORT

FLIGHT TICKET BOOKING AUTOMATION

Submitted By

PACHAVA ANUNYA

Project Guide :

NAVIN CK

SDET Mentor

INDEX

1. ABSTRACT

Automation testing plays a critical role in improving software quality and reducing manual testing effort. This project automates the complete flight booking process available in the BlazeDemo application using Selenium WebDriver and Java.

The framework follows the Page Object Model (POM) design pattern and utilizes TestNG for test execution, Apache POI for reading test data from Excel files, and Maven for dependency management. The automation script performs the complete user journey from searching flights to booking confirmation while validating all critical checkpoints.

2. PROBLEM STATEMENT

Automate the end-to-end flight booking flow on the Simple Travel Agency application.
https://blazedemo.com/

Task

Using Selenium WebDriver, write an automation script to perform the following actions in sequence:

1. Navigate to the Simple Travel Agency application
2. Select a departure city from the dropdown
3. Select a destination city from the dropdown
4. Click on the "Find Flights" button
5. Verify that a list of available flights is displayed
6. Select one flight by clicking "Choose This Flight"
7. Verify that the Purchase Flight page is displayed with flight details and total cost
8. Enter valid personal details required for booking
9. Enter valid payment details
10. Click on the "Purchase Flight" button
11. Verify that the flight booking is completed successfully and a confirmation message is displayed

Input Constraints

Departure and destination cities must be selected before searching for flights
All mandatory personal and payment fields must be filled
Only one flight should be selected per execution

Output
Display a confirmation that the flight booking was completed successfully

Notes
Use Selenium WebDriver
Use Java programming language
The automation should follow the exact user joumey without skipping steps

3. OBJECTIVES

The primary objectives of this project are:

• Automate the complete flight booking workflow

• Validate business functionality

• Reduce manual intervention

• Improve execution speed

• Demonstrate Selenium automation concepts

• Implement Data-Driven Testing

• Generate execution evidence through screenshots

4. SCOPE OF PROJECT

The project covers:

• Functional testing of flight booking process

• Data-driven testing using Excel

• Validation of booking confirmation

• Screenshot generation

• Browser automation using Chrome

The project does not cover:

• Cross-browser testing

• Performance testing

• API testing

• Security testing

5. TECHNOLOGIES USED

Technology	Purpose
Java	Programming Language
Selenium WebDriver	Browser Automation
TestNG	Test Framework
Maven	Build Management
Apache POI	Excel Data Handling
WebDriverManager	Driver Management
Eclipse IDE	Development Environment
Chrome Browser	Test Execution
Commons IO	Screenshot Support
Log4j	Logging Framework

6. SYSTEM REQUIREMENTS

Hardware Requirements

• Processor: Intel i3 or above

• RAM: 4 GB minimum

• Storage: 2 GB free space

Software Requirements

• Windows 10/11

• Java JDK 17 or above

• Eclipse IDE

• Maven

• Google Chrome

• Selenium WebDriver

7. FRAMEWORK DESIGN

The framework follows:

PAGE OBJECT MODEL (POM)

Advantages:

• Better maintainability

• Reusability of code

• Improved readability

• Easy scalability

• Separation of test logic and page objects

8. PROJECT ARCHITECTURE

FlightTicketBookingAutomation

src/main/java

pages

- HomePage.java
- ReservePage.java
- PurchasePage.java
- ConfirmationPage.java

utilities

- ExcelUtils.java
- ScreenshotUtil.java

src/test/java

tests

- FlightBookingTest.java

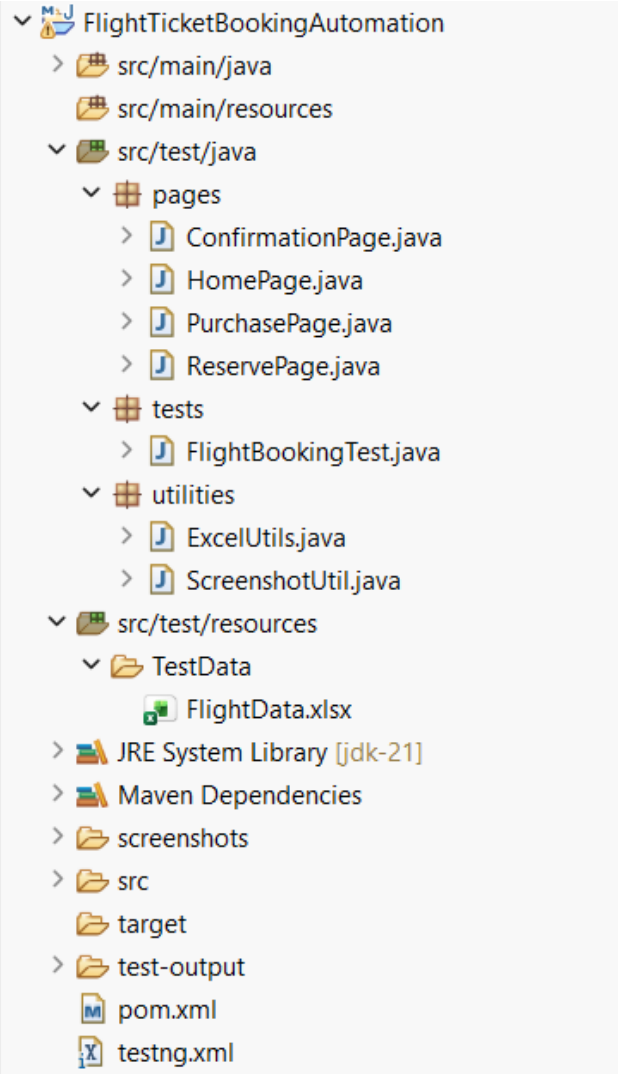
resources

TestData

- FlightData.xlsx

Configuration Files

- pom.xml
- testng.xml



9. PAGE OBJECT MODEL IMPLEMENTATION

HomePage.java

Functions:

- Select departure city
- Select destination city
- Click Find Flights button

ReservePage.java

Functions:

- Verify flight availability
- Select flight

PurchasePage.java

Functions:

- Enter passenger information
- Enter payment information
- Purchase flight

ConfirmationPage.java

Functions:

- Capture confirmation message
- Validate successful booking

10. TEST DATA MANAGEMENT

Test data is maintained externally using Excel.

	A	B	C	D	E	F	G	H	I	J	K
1	**FromCity**	**ToCity**	**Name**	**Address**	**City**	**State**	**Zip**	**CardNumber**	**Month**	**Year**	**NameOnCard**
2	Boston	London	Anunya	Hyderabad	Hyderabad	Telangana	500001	123456789012	12	2028	Anunya
3								3450			
4											

11. AUTOMATION IMPLEMENTATION

pom.xml

```
   http://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation with catalog)
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>blazedemo</groupId>
  <artifactId>FlightTicketBookingAutomation</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>FlightTicketBookingAutomation</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>

    <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.44.0</version>
        <scope>compile</scope>
    </dependency>


    <!-- Source: https://mvnrepository.com/artifact/io.github.bonigarcia/webdrivermanager -->
    <dependency>
        <groupId>io.github.bonigarcia</groupId>
        <artifactId>webdrivermanager</artifactId>
        <version>6.3.3</version>
        <scope>compile</scope>
    </dependency>

    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.12.0</version>
        <scope>test</scope>
    </dependency>

    <!-- Source: https://mvnrepository.com/artifact/org.apache.poi/poi-ooxml -->
    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
        <version>5.5.1</version>
        <scope>compile</scope>
    </dependency>

    <!-- Commons IO -->
    <dependency>
        <groupId>commons-io</groupId>
        <artifactId>commons-io</artifactId>
        <version>2.18.0</version>
    </dependency>

```

```
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>2.24.3</version>
    </dependency>

    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.24.3</version>
    </dependency>

  </dependencies>
</project>
```

testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>

Bind to grammar/schema...
<suite name="FlightBookingSuite">

    <test name="FlightBookingTest">

        <classes>

            <class name="tests.FlightBookingTest"/>

        </classes>

    </test>

</suite>
```

HomePage.java

```java
package pages;

import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.Select;
//Page Object class for Home Page
public class HomePage {

    WebDriver driver;
    // Constructor
    public HomePage(WebDriver driver)
    {
        this.driver = driver;
    }
    // Locators
    By fromCity = By.name("fromPort");
    By toCity = By.name("toPort");
    By findFlights = By.xpath("//input[@value='Find Flights']");
    // Select departure city from dropdown
    public void selectDepartureCity(String city)
    {
        Select select = new Select(driver.findElement(fromCity));
        select.selectByVisibleText(city);
    }
    // Select destination city from dropdown
    public void selectDestinationCity(String city)
    {
        Select select = new Select(driver.findElement(toCity));
        select.selectByVisibleText(city);
    }
    // Click Find Flights button
    public void clickFindFlights()
    {
        driver.findElement(findFlights).click();
    }
}
```

ReservePage.java

```java
package pages;

import java.util.List;
import org.openqa.selenium.*;

// Page Object class for Flight Search Results Page
public class ReservePage {

    WebDriver driver;
    // Constructor
    public ReservePage(WebDriver driver) {
        this.driver = driver;
    }

    // Locator for available flights table rows
    By flights = By.xpath("//table/tbody/tr");

    // Locator for first available flight
    By chooseFlight = By.xpath("(//input[@value='Choose This Flight'])[1]");

    // Returns number of flights available
    public int getFlightCount()
    {
        List<WebElement> rows = driver.findElements(flights);
        return rows.size();
    }

    // Select first available flight
    public void selectFlight()
    {
        driver.findElement(chooseFlight).click();
    }
}
```

PurchasePage.java

```java
package pages;

import org.openqa.selenium.*;
//Page Object class for Purchase Flight Page
public class PurchasePage {

    WebDriver driver;
    public PurchasePage(WebDriver driver)
    {
        this.driver = driver;
    }
    // Passenger Information Locators
    By name = By.id("inputName");
    By address = By.id("address");
    By city = By.id("city");
    By state = By.id("state");
    By zipCode = By.id("zipCode");
    // Payment Information Locators
    By cardNumber = By.id("creditCardNumber");
    By cardMonth = By.id("creditCardMonth");
    By cardYear = By.id("creditCardYear");
    By nameOnCard = By.id("nameOnCard");
    // Purchase button locator
    By purchaseButton = By.xpath("//input[@value='Purchase Flight']");

    // Enter passenger details
    public void enterPassengerDetails(String pname, String paddress, String pcity, String pstate, String pzip)
    {
        driver.findElement(name).sendKeys(pname);
        driver.findElement(address).sendKeys(paddress);
        driver.findElement(city).sendKeys(pcity);
        driver.findElement(state).sendKeys(pstate);
        driver.findElement(zipCode).sendKeys(pzip);
    }
    // Enter payment details
    public void enterPaymentDetails(String cardNo, String month, String year, String cardHolder)
    {
        driver.findElement(cardNumber).clear();
        driver.findElement(cardNumber).sendKeys(cardNo);

        driver.findElement(cardMonth).clear();
        driver.findElement(cardMonth).sendKeys(month);

        driver.findElement(cardYear).clear();
        driver.findElement(cardYear).sendKeys(year);

        driver.findElement(nameOnCard).sendKeys(cardHolder);
    }
    // Click Purchase Flight button
    public void clickPurchaseFlight() {
        driver.findElement(purchaseButton).click();
    }
}
```

ConfirmationPage.java

```java
package pages;

import org.openqa.selenium.*;

//Page Object class for Booking Confirmation Page
public class ConfirmationPage {

    WebDriver driver;
    // Constructor to initialize WebDriver
    public ConfirmationPage(WebDriver driver)
    {
        this.driver = driver;
    }
    // Locator for booking confirmation message
    By confirmationMessage = By.tagName("h1");
    // Returns confirmation message displayed after booking
    public String getConfirmationMessage()
    {
        return driver.findElement(confirmationMessage).getText();
    }
}
```

FlightBookingTest.java

```java
package tests;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.*;
import io.github.bonigarcia.wdm.WebDriverManager;
import pages.*;
import utilities.ExcelUtils;
import utilities.ScreenshotUtil;

Run All
public class FlightBookingTest {

    WebDriver driver;
    WebDriverWait wait;
    @BeforeMethod
    public void setup() {
        // Browser setup before every test execution
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }
```

```
    public void pause() {
        try {
            Thread.sleep(3000);
        }
        catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    // End-to-End Flight Booking Test
    @Test
    Run | Debug
    public void bookFlight() throws Exception {

        // Launch application
        System.out.println("Application Launched Successfully");
        driver.get("https://blazedemo.com");

        // Wait for page to load
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//select[@name='fromPort']")));

        // Read test data from Excel
        String fromCity = ExcelUtils.getCellData(1, 0);
        String toCity = ExcelUtils.getCellData(1, 1);
        String name = ExcelUtils.getCellData(1, 2);
        String address = ExcelUtils.getCellData(1, 3);
        String city = ExcelUtils.getCellData(1, 4);
        String state = ExcelUtils.getCellData(1, 5);
        String zip = ExcelUtils.getCellData(1, 6);
        String cardNumber = ExcelUtils.getCellData(1, 7);
        String month = ExcelUtils.getCellData(1, 8);
        String year = ExcelUtils.getCellData(1, 9);
        String cardHolder = ExcelUtils.getCellData(1, 10);


        HomePage home = new HomePage(driver);

        // Select cities and search flights
        home.selectDepartureCity(fromCity);
        System.out.println("Departure City Selected");

        home.selectDestinationCity(toCity);
        System.out.println("Destination City Selected");

        // Verify flights are available
        home.clickFindFlights();
        System.out.println("Find Flights Clicked");

        // Wait until flight table is displayed
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//table[@class='table']")));

        ReservePage reserve = new ReservePage(driver);

        int flights = reserve.getFlightCount();
        System.out.println("Flights Available : " + flights);

        Assert.assertTrue(flights > 0);
        reserve.selectFlight();
        System.out.println("Flight Selected");

        // Wait until Purchase Page loads
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("inputName")));

        System.out.println("Purchase Page Displayed");

        PurchasePage purchase = new PurchasePage(driver);
```

```
95
96          // Enter passenger details
97          purchase.enterPassengerDetails(name, address, city, state, zip);
98          System.out.println("Passenger Details Entered");
99
100         // Enter payment details
101         purchase.enterPaymentDetails(cardNumber, month, year, cardHolder);
102         System.out.println("Payment Details Entered");
103
104         // Complete booking
105         purchase.clickPurchaseFlight();
106         System.out.println("Purchase Flight Clicked");
107
108         // Wait for Confirmation Page
109         wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//h1[contains(text(),'Thank you')]")));
110
111         // Validate confirmation message
112         ConfirmationPage confirm = new ConfirmationPage(driver);
113         String message = confirm.getConfirmationMessage();
114
115         Assert.assertEquals(message, "Thank you for your purchase today!");
116         System.out.println("Booking Successful");
117         System.out.println("Confirmation Message : " + message);
118
119         // Capture screenshot
120         String screenshot = ScreenshotUtil.captureScreenshot(driver, "Booking_Success");
121         System.out.println("Screenshot Saved : " + screenshot);
122     }
123
```

```
123
124     // Close browser after execution
125     @AfterMethod
126     public void tearDown() {
127         if(driver != null)
128         {
129             driver.quit();
130             System.out.println("Browser Closed Successfully");
131         }
132     }
133 }
```

ExcelUtils.java

```
1  package utilities;
2
3  import java.io.InputStream;
4  import org.apache.poi.ss.usermodel.*;
5  import org.apache.poi.xssf.usermodel.XSSFWorkbook;
6
7  //Utility class for reading Excel data
8  public class ExcelUtils {
9
10     // Returns cell value from Excel sheet
11     public static String getCellData(int row, int col) throws Exception {
12
13         InputStream is = ExcelUtils.class.getClassLoader()
14                 .getResourceAsStream("TestData/FlightData.xlsx");
15
16         Workbook wb = new XSSFWorkbook(is);
17         Sheet sheet = wb.getSheet("Sheet1");
18         String value = sheet.getRow(row)
19                             .getCell(col)
20                             .toString();
21         wb.close();
22         return value;
23     }
24 }
```

ScreenshotUtil.java

```java
package utilities;

import java.io.File;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.apache.commons.io.FileUtils;
import org.openqa.selenium.*;

// Utility class for screenshot capture
public class ScreenshotUtil {

    // Captures screenshot and returns saved path
    public static String captureScreenshot(
            WebDriver driver,
            String fileName) throws Exception {

        // Generate unique timestamp
        String timestamp = new SimpleDateFormat("yyyyMMdd_HHmmss")
                        .format(new Date());

        // Screenshot file path
        String path = System.getProperty("user.dir")
                        + "\\screenshots\\"
                        + fileName + "_"
                        + timestamp + ".png";

        // Capture screenshot
        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        // Save screenshot
        FileUtils.copyFile(src, new File(path));
        return path;
    }
}
```

12. TEST EXECUTION RESULTS

```
Application Launched Successfully
Departure City Selected
Destination City Selected
Find Flights Clicked
Flights Available : 5
Flight Selected
Purchase Page Displayed
Passenger Details Entered
Payment Details Entered
Purchase Flight Clicked
Booking Successful
Confirmation Message : Thank you for your purchase today!
Screenshot Saved : C:\Wipro_selenium_java\FlightTicketBookingAutomation\screenshots\Booking_Success_20260615_125036.png
Browser Closed Successfully
PASSED: tests.FlightBookingTest.bookFlight

===============================================
    Default test
    Tests run: 1, Failures: 0, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
===============================================
```

13. SCREENSHOTS

Screenshot Name:

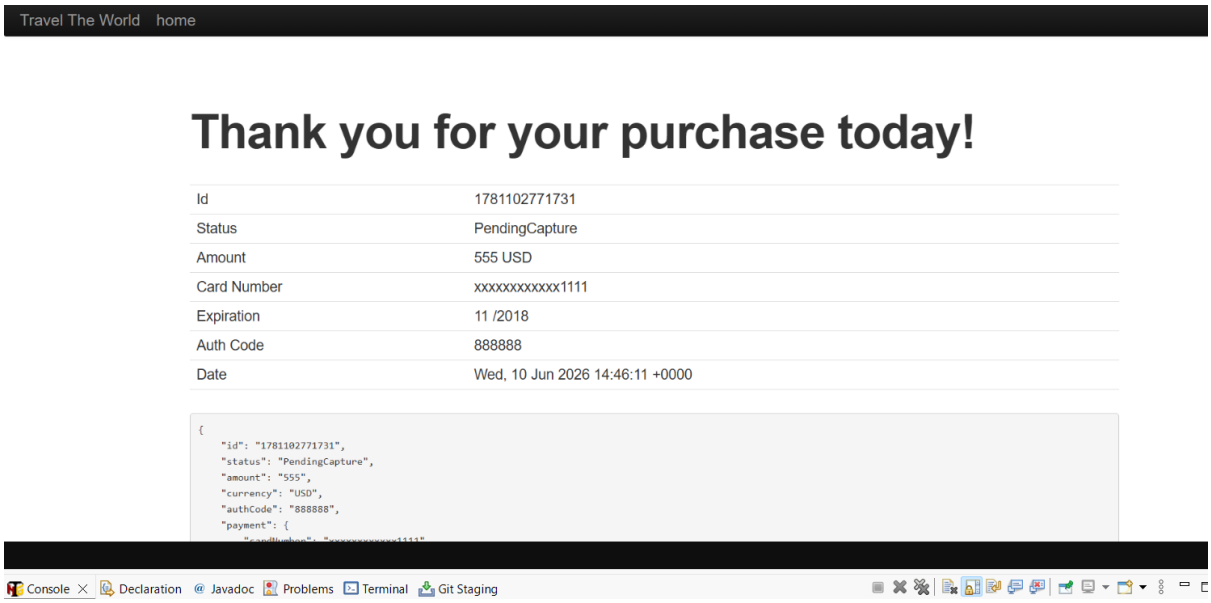
Booking_Success_20260610_201610.png

Location:

C:\Wipro_selenium_java\FlightTicketBookingAutomation\screenshots\

Purpose:

Provides evidence of successful test execution.



GitHub Repository:

Repository URL:

https://github.com/anunya7603/Wipro_Capstone_Project

GitHub is used for:

- Source Code Management
- Version Control
- Collaboration
- Jenkins Integration

14. CHALLENGES FACED

• Locating web elements accurately

• Managing page navigation

• Reading data from Excel

• Implementing Page Object Model

• Capturing screenshots dynamically

• Handling synchronization between pages

15. FUTURE ENHANCEMENTS

• Extent Reports Integration

• Allure Reporting

• Cross Browser Testing

• Jenkins CI/CD Integration

• Parallel Execution

• Database Validation

• Docker Integration

• Cloud Execution using Selenium Grid

• Advanced Logging Framework

16. CONCLUSION

The Flight Ticket Booking Automation project was successfully developed and executed using Selenium WebDriver with Java. The framework automates the complete booking workflow from flight search to booking confirmation while validating all critical checkpoints.

The project demonstrates practical implementation of Selenium automation concepts including Page Object Model, TestNG framework, Data-Driven Testing, Screenshot Capture, Assertions, and Maven dependency management.

Hence, the automation framework successfully fulfills all requirements specified in the problem statement.